

Relatório Técnico: Implementação da DSL VSSScript no Projeto FHOBots

1. Objetivo do Documento

Este documento descreve, passo a passo e em alto nível de detalhe, todas as alterações realizadas no projeto para implementar uma DSL (Domain Specific Language, ou Linguagem de Domínio Específico) voltada à simplificação da definição de estratégias e regras táticas da máquina de estados dos robôs da VSSLeague.

A implementação foi baseada no documento "DSL - FHOBots.pdf", que propunha a criação de uma linguagem chamada VSSScript com suporte a:

- definição de estratégias com STRATEGY;
- condições usando IF ... THEN ... END;
- condições baseadas em bola, robôs e áreas do campo;
- ações como GO_TO, KICK, DEFEND, SUPPORT, HOLD e MARK;
- análise léxica;
- análise sintática;
- análise semântica;
- interpretação da estratégia.

O resultado final é uma DSL integrada ao projeto C++ existente, usando Flex para o analisador léxico e Bison/Yacc para o analisador sintático.

2. Situação Inicial do Projeto

Antes das alterações, o projeto já possuía uma arquitetura de estados implementada em C++.

Os principais elementos existentes eram:

- Robot: classe base dos robôs;
- MachineState: máquina de estados responsável por armazenar e trocar estados;
- State: interface base para cada estado concreto;
- estados específicos para atacante, defensor e goleiro;
- WorldModel: funções de consulta do mundo, como bola em área de ataque, robô perto da bola, etc.;
- Global: armazenamento global de robôs, bola, áreas e comunicação.

O fluxo original era:

```
Robot::updateRobot()  
-> MachineState::think()  
-> State::doActions()  
-> State::checkConditions()  
-> MachineState::setState(...)
```

Cada estado definia suas próprias transições dentro de checkConditions(). Isso funcionava, mas as regras táticas ficavam espalhadas por vários arquivos .cpp.

Exemplo do problema:

- para alterar uma regra do atacante, era necessário editar arquivos em strategy/attacker/;
- para alterar uma regra do defensor, arquivos em strategy/defender/;
- para alterar uma regra do goleiro, arquivos em strategy/goalkeeper/.

A DSL foi criada para permitir que uma estratégia pudesse ser escrita em um arquivo texto, mais próximo da

linguagem do domínio:

```
IF BALL IN ATTACK_AREA THEN
  ATTACKER KICK GOAL
  DEFENDER HOLD DEFENSE
  GOALKEEPER DEFEND GOAL
END
```

3. Decisão de Arquitetura

Foram consideradas três possibilidades:

1. Criar uma DSL que controlasse diretamente PWM e motores.
2. Criar uma DSL que gerasse código C++.
3. Criar uma DSL tática que selecionasse estados e ações de alto nível.

A opção escolhida foi a terceira.

Motivo:

- o projeto já possuía estados calibrados;
- os estados já conheciam controle, movimento e comunicação;
- a DSL deveria simplificar a estratégia, não substituir o controle físico;
- a proposta do documento explicitamente deixava PID, hardware real e simulação física detalhada fora do escopo.

Assim, a DSL foi implementada como uma camada acima da máquina de estados.

Ela interpreta comandos como:

```
ATTACKER GO_TO BALL
```

e converte para comportamentos já existentes, por exemplo:

```
ATTACKER -> estado "seeking"
```

4. Novos Arquivos Criados

Foram criados os seguintes arquivos:

```
strategy/dsl/VssScript.hpp
strategy/dsl/VssScript.cpp
strategy/dsl/VssParser.y
strategy/dsl/VssLexer.l
strategy/dsl/README.md
strategy/scripts/default.vss
docs/dsl_implementation_report.md
```

Também são gerados automaticamente durante a build:

```
strategy/dsl/VssParser.cpp
strategy/dsl/VssParser.hpp
strategy/dsl/VssLexer.cpp
```

Esses arquivos gerados não devem ser editados manualmente.

5. Alterações no .gitignore

Arquivo alterado:

```
.gitignore
```

Antes, o .gitignore ignorava apenas objetos compilados e o bin/Ério:

```
bin/**/* .o
bin/* .o
fhobotsTeam
```

Depois, foram adicionados os arquivos gerados por Bison e Flex:

```
strategy/dsl/VssParser.cpp
strategy/dsl/VssParser.hpp
strategy/dsl/VssLexer.cpp
```

Motivo:

- VssParser.cpp e VssParser.hpp são gerados a partir de VssParser.y;
- VssLexer.cpp Ø gerado a partir de VssLexer.l;
- versionar arquivos gerados aumenta ruído no repositório;
- a fonte real da gramÆtica Ø o .y;
- a fonte real do lexer Ø o .l.

6. Alterações no Makefile

Arquivo alterado:

```
Makefile
```

6.1 Inclusão de fontes geradas

Foi adicionada a variÆvel:

```
DSL_GENERATED=strategy/dsl/VssParser.cpp strategy/dsl/VssLexer.cpp
```

Depois, a lista de fontes C++ passou a incluir explicitamente esses arquivos gerados:

```
CPP_SRC=$(filter-out $(DSL_GENERATED),$(wildcard *.cpp) $(wildcard */*.cpp) $(wildcard */**/*.cpp)) $(DSL_GENERATED)
```

Motivo:

- os arquivos VssParser.cpp e VssLexer.cpp não existem antes da build;
- se o Makefile dependesse apenas de wildcard, eles poderiam ficar fora da compilação;
- incluindo explicitamente DSL_GENERATED, garantimos que o Makefile sabe que eles fazem parte do projeto.

6.2 Regra para gerar parser com Bison

Foi adicionada a regra:

```
strategy/dsl/VssParser.cpp strategy/dsl/VssParser.hpp: strategy/dsl/VssParser.y
    bison -d -o strategy/dsl/VssParser.cpp strategy/dsl/VssParser.y
```

Essa regra diz:

- entrada: VssParser.y;
- saída: VssParser.cpp e VssParser.hpp;
- ferramenta: bison;
- flag -d: gera também o header com tokens e tipos.

6.3 Regra para gerar lexer com Flex

Foi adicionada a regra:

```
strategy/dsl/VssLexer.cpp: strategy/dsl/VssLexer.l strategy/dsl/VssParser.hpp  
flex -o strategy/dsl/VssLexer.cpp strategy/dsl/VssLexer.l
```

Essa regra diz:

- entrada: VssLexer.l;
- dependência adicional: VssParser.hpp;
- saída: VssLexer.cpp;
- ferramenta: flex.

O lexer depende do header do parser porque precisa conhecer os tokens gerados pelo Bison.

6.4 Limpeza de arquivos gerados

No alvo clean, foram adicionados:

```
rm -f strategy/dsl/VssParser.cpp strategy/dsl/VssParser.hpp strategy/dsl/VssLexer.cpp
```

Isso permite uma build limpa do zero.

6.5 Diretório de objetos da DSL

No alvo bin_dir, foi adicionado:

```
bin/strategy/dsl
```

Sem isso, a compilação falhava ao tentar criar:

```
bin/strategy/dsl/VssScript.o  
bin/strategy/dsl/VssParser.o  
bin/strategy/dsl/VssLexer.o
```

7. Alterações em MachineState

Arquivos alterados:

```
strategy/MachineState.hpp  
strategy/MachineState.cpp
```

7.1 Antes

Antes, MachineState::setState tinha retorno void:

```
void setState(const std::string nameState);
```

E a implementação usava diretamente:

```
_activeState = _states[nameState];
```

Isso tinha dois problemas:

1. se o estado não existisse, o operador [] criaria uma entrada nula no mapa;
2. trocar para o mesmo estado repetidamente chamava exitActions() e entryActions() sem necessidade.

7.2 Depois

O método passou a retornar bool:

```
bool setState(const std::string nameState);
```

Agora ele:

1. procura o estado com find;
2. retorna false se o estado não existir;
3. evita reentrar no mesmo estado;
4. retorna true quando a troca é válida.

Fluxo atual:

```
setState("seeking")
-> procura "seeking" no mapa
-> se não existir, imprime erro e retorna false
-> se já for o estado atual, retorna true
-> executa exitActions do estado antigo
-> troca o ponteiro ativo
-> executa entryActions do novo estado
-> retorna true
```

7.3 Por que isso foi necessário?

A DSL permite escrever:

```
ATTACKER USE seeking
```

Se o usuário escrever:

```
ATTACKER USE estado_inexistente
```

o sistema não deve quebrar silenciosamente.

Além disso, a DSL é executada em loop. Se a bola continuar na área de ataque, a regra pode ser verdadeira por vários frames consecutivos. Sem a proteção contra reentrada, o robô chamaria entryActions() repetidamente no mesmo estado, o que poderia reinicializar variáveis internas e gerar comportamento instável.

7.4 Ajuste em currentState

Também foi adicionada proteção em:

```
const std::string MachineState::currentState()
```

Agora, se _activeState for nulo, retorna string vazia.

Isso evita acesso inválido caso o estado ainda não tenha sido inicializado.

8. Alterações em Robot

Arquivos alterados:

```
model/Robot.hpp
model/Robot.cpp
```

8.1 Novos métodos públicos

Foram adicionados:

```
bool setState(const std::string& stateName);
std::string currentState();
```

8.2 Objetivo

Antes, `_machineState` era protegido dentro de `Robot`, então apenas classes derivadas ou a própria classe podiam trocar estados.

A DSL precisava aplicar ações em robôs globais:

```
Global::attacker
Global::defender
Global::goalkeeper
```

Para isso, era necessário expor uma API controlada:

```
robot.setState("seeking");
```

Essa API encapsula a máquina de estados e mantém o restante do código sem acesso direto ao `_machineState`.

9. Novo Arquivo: `VssScript.hpp`

Arquivo criado:

```
strategy/dsl/VssScript.hpp
```

Esse arquivo define a AST da DSL e a interface pública do interpretador.

10. AST: `rvore Sintática Abstrata

A AST é a representação em memória da estratégia depois do parsing.

Texto da DSL:

```
IF BALL IN ATTACK_AREA THEN
  ATTACKER KICK GOAL
END
```

Representação conceitual:

```
Statement
  Condition: BallInArea(Attack)
  Actions:
    Action: Attacker KickGoal
```

10.1 RobotRole

```
enum class RobotRole {
  Attacker,
  Defender,
  Goalkeeper
};
```

Representa o papel do robô na estratégia.

A DSL aceita:

```
ATTACKER
```

```
DEFENDER
GOALKEEPER
ROBOT 1
ROBOT 2
ROBOT 3
```

Mapeamento:

```
ROBOT 1 -> GOALKEEPER
ROBOT 2 -> DEFENDER
ROBOT 3 -> ATTACKER
```

Esse mapeamento segue a configuração do projeto, em que:

- goleiro usa posição de mensagem 0;
- defensor usa posição de mensagem 1;
- atacante usa posição de mensagem 2.

Como a linguagem do documento usa ROBOT 1, ROBOT 2 e ROBOT 3, a DSL aceita essa forma, mas também aceita nomes mais claros.

10.2 FieldArea

```
enum class FieldArea {
    Attack,
    Defense,
    Center
};
```

Representa as Áreas:

```
ATTACK_AREA
DEFENSE_AREA
CENTER_AREA
```

10.3 ConditionType

```
enum class ConditionType {
    Always,
    KeySpace,
    BallInArea,
    RobotHasBall,
    RobotNearBall,
    RobotInArea
};
```

Condições suportadas:

```
KEY SPACE
BALL IN ATTACK_AREA
BALL IN DEFENSE_AREA
BALL IN CENTER_AREA
ATTACKER HAS_BALL
ATTACKER NEAR BALL
ATTACKER IN ATTACK_AREA
```

10.4 ActionType

```
enum class ActionType {
    UseState,
    GoToBall,
    GoToPosition,
```

```

    KickGoal,
    KickCenter,
    DefendGoal,
    HoldDefense,
    MarkBall,
    Support
};

```

Ações suportadas:

```

ATTACKER USE seeking
ATTACKER GO_TO BALL
ATTACKER GO_TO (320, 240)
ATTACKER KICK GOAL
ATTACKER KICK CENTER
GOALKEEPER DEFEND GOAL
DEFENDER HOLD DEFENSE
DEFENDER MARK BALL
DEFENDER SUPPORT CENTER

```

10.5 SourceLocation

```

struct SourceLocation {
    int line = 1;
    int column = 1;
};

```

Guarda linha e coluna do comando no arquivo .vss.

Isso permite mensagens como:

```

line 3, column 12: ATTACKER does not have state 'nonexistent'

```

10.6 Condition

```

struct Condition {
    ConditionType type = ConditionType::Always;
    RobotRole robot = RobotRole::Attacker;
    FieldArea area = FieldArea::Center;
    SourceLocation location;
};

```

Representa uma condição.

Exemplo:

```

BALL IN ATTACK_AREA

```

vira:

```

type = BallInArea
area = Attack

```

10.7 Action

```

struct Action {
    RobotRole robot = RobotRole::Attacker;
    ActionType type = ActionType::UseState;
    std::string value;
    int x = 0;
    int y = 0;
    SourceLocation location;
};

```



```
};
```

Representa uma ação.

Exemplo:

```
ATTACKER GO_TO (320, 240)
```

vira:

```
robot = Attacker  
type = GoToPosition  
x = 320  
y = 240
```

10.8 Statement

```
struct Statement {  
    Condition condition;  
    std::vector<Action> actions;  
};
```

Representa uma regra.

Cada regra possui:

- uma condição;
- uma ou mais ações.

10.9 Strategy

```
struct Strategy {  
    std::string name;  
    std::vector<Statement> statements;  
};
```

Representa o programa completo.

11. Novo Lexer com Flex

Arquivo criado:

```
strategy/dsl/VssLexer.l
```

O lexer transforma texto em tokens.

Exemplo:

```
IF BALL IN ATTACK_AREA THEN
```

vira:

```
IF  
BALL  
IN  
ATTACK_AREA  
THEN
```

11.1 Opções do Flex

```
%option noyywrap nounput noinput caseless yylineno
```

Significado:

- noyywrap: evita necessidade de implementar yywrap;
- nounput: não gera função unput, que não é usada;
- noinput: não gera função input, que não é usada;
- caseless: aceita palavras-chave sem diferenciar maiúsculas/minúsculas;
- yylineno: ativa contagem de linhas.

11.2 Tracking de coluna

Flex já rastreia linha com yylineno, mas não coluna automaticamente.

Por isso foi criada a variável:

```
int vss_yycolumn = 1;
```

E o macro:

```
#define YY_USER_ACTION ...
```

Esse macro roda a cada token reconhecido e atualiza:

- yylloc.first_line;
- yylloc.first_column;
- yylloc.last_line;
- yylloc.last_column.

Esses dados são usados pelo Bison para erros com localização.

11.3 Comentários

Comentários começam com #:

```
# isto é um comentário
```

Regra:

```
"#" . * ;
```

11.4 Tokens de palavras-chave

O lexer reconhece:

```
STRATEGY  
IF  
THEN  
END  
BALL  
ROBOT  
GOAL  
KEY  
SPACE  
IN  
HAS_BALL  
NEAR  
USE  
GO_TO  
KICK  
DEFEND  
SUPPORT
```

```
HOLD
MARK
ATTACKER
DEFENDER
GOALKEEPER
ATTACK_AREA
DEFENSE_AREA
DEFENCE_AREA
CENTER_AREA
LEFT
RIGHT
CENTER
```

DEFENCE_AREA também foi aceito como alternativa a DEFENSE_AREA.

11.5 Identificadores

Regra:

```
[A-Za-z_][A-Za-z0-9_]*
```

Usada principalmente para:

```
STRATEGY default
ATTACKER USE seeking
```

11.6 Números

Regra:

```
[0-9]+
```

Usada para:

```
ROBOT 1
GO_TO (320, 240)
```

12. Novo Parser com Bison/Yacc

Arquivo criado:

```
strategy/dsl/VssParser.y
```

O parser valida a estrutura da linguagem e monta a AST.

13. Gramática Implementada

Resumo em EBNF:

```
program      ::= "STRATEGY" identifier statement_list
statement_list ::= statement | statement_list statement
statement    ::= if_statement | action_statement
if_statement ::= "IF" condition "THEN" action_list "END"
action_statement ::= action

condition    ::= "KEY" "SPACE"
               | "BALL" "IN" field_area
               | robot "HAS_BALL"
               | robot "NEAR" "BALL"
               | robot "IN" field_area
```

```

action                ::= robot "USE" identifier
                        | robot "GO_TO" "BALL"
                        | robot "GO_TO" "(" number "," number ")"
                        | robot "KICK" "GOAL"
                        | robot "KICK" "CENTER"
                        | robot "DEFEND" "GOAL"
                        | robot "HOLD" "DEFENSE"
                        | robot "MARK" "BALL"
                        | robot "SUPPORT" support_area

robot                  ::= "ATTACKER" | "DEFENDER" | "GOALKEEPER" | "ROBOT" number
field_area             ::= "ATTACK_AREA" | "DEFENSE_AREA" | "CENTER_AREA"
support_area           ::= "LEFT" | "RIGHT" | "CENTER"

```

14. Como o Parser Monta a AST

Exemplo de regra:

```

condition:
  BALL IN field_area
  {
    $$ = new vsscript::Condition();
    $$->type = vsscript::ConditionType::BallInArea;
    $$->area = $3;
    $$->location = { @1.first_line, @1.first_column };
  }
;

```

Interpretação:

- quando o parser encontra BALL IN <area>;
- cria um Condition;
- define o tipo como BallInArea;
- salva a Área;
- salva linha/coluna do token BALL.

Exemplo para ação:

```
robot GO_TO '(' NUMBER ',' NUMBER ')'
```

Essa regra cria:

```

Action {
  robot = <robot escolhido>;
  type = GoToPosition;
  x = numero_1;
  y = numero_2;
}

```

15. Tratamento de ROBOT 1, 2 e 3

No parser:

```

ROBOT 1 -> Goalkeeper
ROBOT 2 -> Defender
ROBOT 3 -> Attacker

```

Se o usuário escrever:

```
ROBOT 4 GO_TO BALL
```

o parser chama erro:

```
invalid robot number: use ROBOT 1, ROBOT 2 or ROBOT 3
```

16. Erros com Linha e Coluna

O Bison usa %locations.

Isso habilita variáveis como:

```
@1.first_line  
@1.first_column  
yylloc.first_line  
yylloc.first_column
```

A função:

```
void yyerror(const char* message)
```

foi implementada para montar mensagens assim:

```
line 3, column 12: syntax error
```

Na análise semântica, o mesmo padrão foi usado:

```
line 3, column 12: ATTACKER does not have state 'nonexistent'
```

17. Novo Interpretador: VssScript.cpp

Arquivo criado:

```
strategy/dsl/VssScript.cpp
```

Esse arquivo faz:

1. carregamento do arquivo .vss;
2. chamada do parser;
3. chamada da análise semântica;
4. descrição textual da estratégia;
5. execução das regras no loop principal.

18. Carregamento da Estratégia

Método:

```
bool VssScript::loadFromFile(const std::string& path)
```

Fluxo:

```
abre arquivo  
-> lê conteúdo para string  
-> chama loadSource(...)
```

Método:

```
bool VssScript::loadSource(const std::string& source)
```

Fluxo:

```
limpa estado anterior
-> parseSource(...)
-> validateStrategy(...)
-> marca _loaded = true
```

Se parsing ou validação falharem, _loaded permanece falso.

19. Análise Semântica

Função:

```
bool validateStrategy(const Strategy& strategy, std::string& error)
```

19.1 Validação do nome da estratégia

Se o nome estiver vazio:

```
strategy name cannot be empty
```

19.2 Validação de estados por robô

Foram definidos os estados válidos por papel.

Atacante:

```
idle
backoff
attacking
seeking
waiting
spinning
align
exit
```

Defensor:

```
idle
backoff
seeking
waiting
kicking
spinning
align
exit
```

Goleiro:

```
idle
seeking
kicking
backoff
moveback
moveforward
spinning
align
waiting
return
exit
retreating
```

Se a DSL tentar usar um estado inexistente:

```
ATTACKER USE nonexistent
```

o erro seria:

```
line 3, column 12: ATTACKER does not have state 'nonexistent'
```

19.3 Validação de ação incoerente

Foi adicionada uma regra semântica:

```
GOALKEEPER KICK CENTER
```

Ø rejeitado.

Motivo:

- o goleiro não possui lógica específica para defesa e chute;
- KICK CENTER Ø mais coerente para atacante/defensor;
- isso demonstra a etapa semântica pedida no documento.

Erro:

```
GOALKEEPER cannot KICK CENTER in current semantic rules
```

20. Avaliação de Condições

Função:

```
bool evaluate(const Condition& condition)
```

20.1 KEY SPACE

```
Global::bufferKeyboard == 32
```

Permite iniciar comportamento ao pressionar espaço.

20.2 BALL IN ATTACK_AREA

Usa:

```
WorldModel::isInAttackArea(Global::ball)
```

20.3 BALL IN DEFENSE_AREA

Usa:

```
WorldModel::isInDefenseArea(Global::ball)
```

20.4 BALL IN CENTER_AREA

Como não havia área central explícita no projeto, foi definida como faixa central do campo:

```
position.x >= 3 * Global::fieldRect.width / 8 &&  
position.x <= 5 * Global::fieldRect.width / 8
```

20.5 ROBOT NEAR BALL

Usa:

```
WorldModel::isNearOf(robot.getPosition(), Global::ball)
```

20.6 ROBOT HAS_BALL

Considera que o robô tem a bola se:

1. está perto da bola;
2. está alinhado com a bola.

Código conceitual:

```
WorldModel::isNearOf(robot.getPosition(), Global::ball) &&  
WorldModel::isAlignedWith(robot.getOrientation(), robotToBall)
```

21. Mapeamento de Ações para Comportamento

Função:

```
std::string mappedState(const Action& action)
```

Mapeamentos principais:

GO_TO BALL	-> seeking
MARK BALL	-> seeking
KICK GOAL	-> attacking para atacante, kicking para outros
KICK CENTER	-> attacking para atacante, kicking para outros
DEFEND GOAL	-> waiting
HOLD DEFENSE	-> waiting
SUPPORT	-> waiting
USE <estado>	-> <estado>

22. Execução de Ações

Função:

```
void executeAction(const Action& action)
```

22.1 Ações baseadas em estado

Para a maioria das ações:

```
robotFor(action.robot).setState(state);
```

Isso reaproveita a máquina de estados existente.

22.2 GO_TO (x,y)

Para posição explícita:

```
Vector2D destination(action.x, action.y);  
Robot& robot = robotFor(action.robot);  
robot.calculatePwmUnivector(destination);  
Global::communication->writeMessage(robot.getPosMessage(), robot.getPwmLeft(), robot.getPwmRight());
```

Isso permite comandos como:

```
ROBOT 3 GO_TO (320, 240)
```


Essa ação é executada diretamente, pois os estados existentes não tinham um estado genérico parametrizado por coordenada.

23. Alterações em main.cpp

Arquivo alterado:

```
main.cpp
```

23.1 Inclusão da DSL

Foi incluído:

```
#include "strategy/dsl/VssScript.hpp"
```

23.2 Novo argumento de estratégia

Foi criado:

```
std::string strategyPath = "strategy/scripts/default.vss";
```

Se o usuário passar um arquivo .vss, ele substitui o padrão:

```
./fhobotsTeam sim minha_estrategia.vss
```

23.3 Modo de checagem

Foram adicionados:

```
--check-strategy  
--print-strategy
```

Uso:

```
./fhobotsTeam --check-strategy strategy/scripts/default.vss
```

Esse modo:

1. carrega o arquivo;
2. faz parsing;
3. faz análise semântica;
4. imprime uma descrição;
5. encerra sem abrir visão, comunicação ou simulador.

Isso é útil para testar a DSL sem depender do ambiente de simulação.

23.4 Carregamento durante execução normal

Após inicializar modelo e estados:

```
vssscript::VssScript strategy;  
if(strategy.loadFromFile(strategyPath)){  
    std::cout << "DSL strategy loaded: " << strategy.name() << " (" << strategyPath << ")" << s  
td::endl;  
}else{  
    std::cout << "DSL strategy disabled: " << strategy.error() << std::endl;  
}
```

Se a estratégia falhar, o programa continua sem DSL.

Isso evita que um erro no arquivo .vss derrube o sistema inteiro durante testes.

23.5 Execução da DSL no loop principal

Depois da visão atualizar o mundo:

```
vision->detectionColors();  
strategy.execute();
```

Isso garante que a DSL avalie a posição atual da bola e dos robôs.

24. Script Padrão Criado

Arquivo criado:

```
strategy/scripts/default.vss
```

Conteúdo:

```
STRATEGY default  
  
IF KEY SPACE THEN  
  ATTACKER USE seeking  
  DEFENDER USE waiting  
  GOALKEEPER USE waiting  
END  
  
IF BALL IN ATTACK_AREA THEN  
  ATTACKER KICK GOAL  
  DEFENDER HOLD DEFENSE  
  GOALKEEPER DEFEND GOAL  
END  
  
IF BALL IN DEFENSE_AREA THEN  
  ATTACKER HOLD DEFENSE  
  DEFENDER GO_TO BALL  
  GOALKEEPER GO_TO BALL  
END  
  
IF BALL IN CENTER_AREA THEN  
  ATTACKER GO_TO BALL  
  DEFENDER SUPPORT CENTER  
  GOALKEEPER DEFEND GOAL  
END
```

Esse arquivo demonstra:

- início por tecla espaço;
- ataque;
- defesa;
- comportamento no centro;
- múltiplos robôs em uma mesma regra;
- comandos de alto nível.

25. Documentação da DSL

Arquivo criado:

```
strategy/dsl/README.md
```

Esse README documenta:

- como compilar;
- como validar uma estratégia;
- como rodar com uma estratégia;
- gramática resumida;
- regras semânticas;
- exemplo de uso.

26. Fluxo Completo da DSL

O fluxo final é:

```
Arquivo .vss
-> Flex (VssLexer.l)
-> tokens
-> Bison/Yacc (VssParser.y)
-> AST (Strategy, Statement, Condition, Action)
-> validateStrategy(...)
-> VssScript::execute()
-> Robot::setState(...) ou movimento direto
-> MachineState
-> State::doActions()
-> Global::communication->writeMessage(...)
```

27. Exemplos de Uso

27.1 Validar estratégia

```
./fhobotsTeam --check-strategy strategy/scripts/default.vss
```

Saída esperada:

```
[OK] Strategy: default
[IF] KEY SPACE
  [ACTION] ATTACKER -> use state seeking
  [ACTION] DEFENDER -> use state waiting
  [ACTION] GOALKEEPER -> use state waiting
...
```

27.2 Rodar no simulador

```
./fhobotsTeam sim strategy/scripts/default.vss
```

Ou:

```
./fhobotsTeam sim
```

Nesse caso, strategy/scripts/default.vss é usado automaticamente.

27.3 Exemplo com posição explícita

```
STRATEGY posicao

IF BALL IN CENTER_AREA THEN
  ATTACKER GO_TO (320, 240)
END
```

27.4 Exemplo com erro semântico

```
STRATEGY invalida

IF BALL IN ATTACK_AREA THEN
  ATTACKER USE nonexistent
END
```

Saída:

```
DSL strategy invalid: line 4, column 12: ATTACKER does not have state 'nonexistent'
```

28. Testes Realizados

Foram executados:

```
make clean
make bin_dir
make
```

Resultado:

```
Build concluída com sucesso.
```

Também foi executado:

```
./fhobotsTeam --check-strategy strategy/scripts/default.vss
```

Resultado:

```
Estratégia carregada, validada e descrita corretamente.
```

Também foi testada uma estratégia inválida com estado inexistente.

Resultado:

```
Erro semântico identificado corretamente com linha e coluna.
```

29. Relação com o Documento Original

29.1 Definir domínio e elementos estratégicos

Atendido.

Foram definidos:

- robôs;
- bola;
- Áreas;
- ações;
- condições;
- estratégia.

29.2 Especificar gramática formal

Atendido.

A gramática está em:

```
strategy/dsl/VssParser.y  
strategy/dsl/README.md
```

29.3 Implementar analisador l xico

Atendido.

Implementado com Flex em:

```
strategy/dsl/VssLexer.l
```

29.4 Implementar analisador sint tico

Atendido.

Implementado com Bison/Yacc em:

```
strategy/dsl/VssParser.y
```

29.5 Implementar verifica es sem nticas b sicas

Atendido.

Implementado em:

```
validateStrategy(...)
```

Valida:

- estados inexistentes;
- rob s v lidos;
- a o incoerente do goleiro.

29.6 Desenvolver interpretador

Atendido.

Implementado em:

```
VssScript::execute()  
executeAction(...)  
evaluate(...)
```

29.7 Demonstrar funcionamento com exemplos pr ticos

Atendido.

Exemplos em:

```
strategy/scripts/default.vss  
strategy/dsl/README.md
```

30. Decis es T cnicas Importantes

30.1 Por que n o controlar tudo pela DSL?

Porque o projeto j  tinha l gica de movimenta  o calibrada nos estados existentes.

A DSL deveria reduzir complexidade estratégica, não substituir controle de baixo nível.

30.2 Por que aceitar ATTACKER além de ROBOT 3?

Porque ATTACKER, DEFENDER e GOALKEEPER são mais legíveis.

Mas ROBOT 1, ROBOT 2, ROBOT 3 foram mantidos para compatibilidade com o documento.

30.3 Por que arquivos gerados são ignorados?

Porque a fonte real é:

```
VssParser.y  
VssLexer.l
```

Os arquivos gerados podem ser recriados com:

```
make
```

30.4 Por que o programa continua se a DSL falhar no modo normal?

Para evitar que um erro de estratégia impeça testes com visão ou comunicação.

Em modo normal:

```
DSL inválida -> DSL desabilitada -> programa continua
```

Em modo check:

```
DSL inválida -> retorna código 1
```

31. Limitações Atuais

Apesar de atender ao documento, existem limitações naturais:

1. Não há operadores booleanos AND, OR, NOT.
2. Não há blocos ELSE.
3. GO_TO (x,y) executa movimento direto, enquanto outras ações usam estados.
4. SUPPORT LEFT/RIGHT/CENTER ainda mapeia genericamente para waiting.
5. A análise semântica é básica, não uma verificação ética completa.
6. Não há hot reload automático do arquivo .vss.

32. Próximas Melhorias Recomendadas

Para evoluir a DSL:

1. Adicionar AND, OR, NOT.
2. Adicionar ELSE.
3. Criar estado parametrizado para GO_TO (x,y).
4. Implementar suporte real para SUPPORT LEFT, SUPPORT RIGHT e SUPPORT CENTER.
5. Adicionar modo de recarregar estratégia durante simulação.
6. Criar testes unitários para parser e semântica.
7. Separar SemanticAnalyzer em arquivo próprio caso cresça.
8. Adicionar saída de debug opcional durante execução:

```
[DSL] BALL IN ATTACK_AREA -> true  
[DSL] ATTACKER KICK GOAL -> attacking
```

33. Conclusão

A implementação atual transforma a proposta conceitual do documento em um recurso funcional dentro do projeto.

Agora o FHOBots possui:

- uma DSL textual;
- lexer com Flex;
- parser com Bison/Yacc;
- AST;
- análise semântica;
- interpretação;
- integração com a máquina de estados existente;
- script padrão;
- documentação;
- modo de validação sem abrir simulador.

Com isso, estratégias básicas podem ser alteradas em arquivos .vss, sem recompilar lógica básica em C++ para cada variação simples.

O projeto passa a seguir o fluxo clássico:

Entrada -> Lexer -> Parser -> AST -> Análise Semântica -> Interpretação

Esse fluxo corresponde diretamente ao objetivo descrito no documento original.